

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN**



**BTVN#3 · Dự agent đặt vé máy bay bằng
LangChain**

Môn học: Kỹ thuật xây dựng hệ thống Agentic AI

Lớp: SE373.R11

GVHD: Ngô Ngọc Đăng Khoa

Nhóm thực hiện: Cá nhân

Sinh viên thực hiện:

Dương Chí Cường

23520195

BTVN#3 · Dựn agent đặt vé máy bay bằng LangChain

Tìm hiểu Langchain, LangGraph.

Khái niệm LangChain là gì và framework này đóng vai trò gì trong việc liên kết Mô hình Ngôn ngữ Lớn với các công cụ nghiệp vụ?

LangChain là một framework mã nguồn mở được thiết kế để đơn giản hóa quá trình xây dựng các ứng dụng dựa trên LLM [1]. Trong bài toán đặt vé máy bay, LangChain đóng vai trò là tầng trừu tượng hóa (Abstraction Layer). Framework này cho phép liên kết mô hình Gemini với danh sách các công cụ (Tools) thông qua cơ chế Function Calling. Bên cạnh đó, LangChain cung cấp khả năng cấu hình chuỗi dự phòng (Fallback Chain), giúp hệ thống tự động chuyển đổi giữa các phiên bản mô hình (ví dụ: chuyển từ phiên bản ưu tiên sang các phiên bản dự phòng khi gặp lỗi hạn ngạch API hoặc sự cố mạng), qua đó đảm bảo tính khả dụng liên tục của Agent.

Giới hạn của kiến trúc chuỗi tuyến tính (Chain) trong LangChain là gì, và tại sao LangGraph lại là công nghệ tiên quyết để quản lý luồng điều khiển dạng đồ thị có trạng thái (Stateful Graph)?

Mặc dù LangChain rất mạnh mẽ trong việc xử lý các chuỗi thao tác tuyến tính, các quy trình thực tế như đặt vé máy bay thường mang tính tuần hoàn (Cyclic) và phụ thuộc nặng nề vào trạng thái (State) của giao dịch [2]. Một chu trình đặt vé yêu cầu Agent phải trải qua nhiều bước: tìm kiếm chuyến bay, lưu trữ thông tin lựa chọn, giữ chỗ tạm thời thu giữ mã đặt chỗ (PNR), và xác nhận thanh toán. LangGraph giải quyết bài toán này bằng cách biểu diễn luồng công việc dưới dạng một đồ thị có hướng (StateGraph). Trạng thái trung tâm (AgentState) đóng vai trò là bộ nhớ chung giữ lại lịch sử tin nhắn, mã PNR hiện tại, bước thực thi, thông tin phiên người dùng và các chỉ số vận hành. Các nút (Nodes) trong đồ thị đại diện cho bước suy luận hoặc thực thi, trong khi các cạnh điều kiện (Conditional Edges) quyết định luồng chuyển dịch dựa trên dữ liệu trạng thái thời gian thực.

Thiết kế công cụ giả lập (Tool Mockup) của chương trình gồm những gì? và giới hạn của nó là?

Hệ thống giả lập được thiết kế gồm bốn công cụ cốt lõi, tương ứng với bốn thao tác nghiệp vụ trong thực tế:

- Tìm kiếm chuyến bay (search_flights): Tiếp nhận thông tin điểm đi (origin), điểm đến (destination) và ngày bay. Công cụ truy vấn cơ sở dữ liệu giả lập để trả về danh sách các chuyến bay khả dụng (như các chuyến bay của Vietnam Airlines,

Vietjet Air giữa SGN và HAN) kèm theo giá vé, khung giờ và số lượng ghế còn trống.

- Kiểm tra chi tiết chuyến bay (check_flight_details): Tiếp nhận mã chuyến bay cụ thể (như VN123) để cung cấp thông tin chi tiết về hạng vé và tình trạng chỗ ngồi hiện tại.
- Giữ chỗ tạm thời (hold_ticket): Yêu cầu mã chuyến bay, tên hành khách và số CCCD/Hộ chiếu. Công cụ kiểm tra số lượng ghế trống, tự động khởi tạo một mã đặt chỗ ngẫu nhiên (PNR) duy nhất, lưu trữ thông tin giao dịch ở trạng thái chờ (HELD) và giảm số ghế khả dụng.
- Xác nhận thanh toán & Xuất vé (confirm_booking): Yêu cầu mã PNR và mã xác thực thanh toán (payment_auth_token). Công cụ thẩm định mã xác thực; nếu hợp lệ, trạng thái giao dịch được cập nhật thành thành công (CONFIRMED), đánh dấu hoàn tất quy trình xuất vé.

1.Cài đặt đủ các lớp harness: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền, bàn giao;

Ràng Buộc Dữ Liệu Đầu Vào (Data Constraints Validation).

Làm thế nào để ngăn chặn Agent xử lý các yêu cầu chứa dữ liệu sai lệch, thiếu hụt hoặc vi phạm quy tắc nghiệp vụ hàng không ngay từ tầng tiếp nhận thông tin?

Trong hệ thống đặt vé máy bay, các lỗi do người dùng nhập thiếu thông tin hoặc LLM trích xuất tham số sai cấu trúc có thể gây ra hàng loạt truy vấn vô nghĩa tới hệ thống giả lập hoặc gây ra lỗi vận hành. Để giải quyết vấn đề này, lớp kiểm soát dữ liệu (FlightDataValidator) kết hợp với lược đồ ràng buộc (BookingConstraintSchema) dựa trên thư viện Pydantic được triển khai. Tầng này hoạt động như một bộ lọc tiền xử lý (Pre-processing Filter): mọi yêu cầu từ người dùng hoặc tham số do Agent khởi tạo bắt buộc phải đi qua trình kiểm tra này trước khi kích hoạt công cụ. Nếu dữ liệu không thỏa mãn các tiêu chí độ dài, kiểu dữ liệu hoặc khoảng giá trị, hệ thống sẽ từ chối thực thi và trả về thông báo lỗi chi tiết để Agent điều chỉnh hành vi.

Bảng cấu hình các ràng buộc dữ liệu đầu vào của hệ thống đặt vé

Trường dữ liệu (Field)	Kiểu dữ liệu	Điều kiện kiểm tra / Ràng buộc	Mục đích nghiệp vụ & Bảo mật
origin	String	Độ dài cố định đúng 3 ký tự (VD: SGN)	Đảm bảo mã sân bay đi đúng chuẩn IATA.

destination	String	Độ dài cố định đúng 3 ký tự (VD: HAN)	Đảm bảo mã sân bay đến đúng chuẩn IATA.
max_budget	Float	Giá trị phải lớn hơn 0 (gt=0)	Xác định hạn mức tài chính tối đa của khách hàng.
passenger_name	String	Độ dài tối thiểu 2 ký tự (min_length=2)	Tránh trường hợp bỏ trống tên hành khách.
passenger_id	String	Độ dài tối thiểu 6 ký tự (min_length=6)	Xạ thực số CCCD/Hộ chiếu hợp lệ theo quy định.

- Triển khai trên code.

```
class BookingConstraintSchema(BaseModel):
    """Mô hình Pydantic kiểm tra tính hợp lệ dữ liệu đầu vào."""
    origin: str = Field(..., min_length=3, max_length=3, description="Mã sân bay đi (VD: SGN)")
    destination: str = Field(..., min_length=3, max_length=3, description="Mã sân bay đến (VD: HAN)")
    max_budget: float = Field(..., gt=0, description="Ngân sách tối đa của khách hàng")
    passenger_name: str = Field(..., min_length=2, description="Tên hành khách")
    passenger_id: str = Field(..., min_length=6, description="Số CCCD/Hộ chiếu")

class FlightDataValidator:
    """Lớp kiểm tra dữ liệu đầu vào theo ràng buộc nghiệp vụ."""
    @staticmethod
    def validate_request(request_data: Dict[str, Any]) -> tuple[bool, Optional[str]]:
        try:
            BookingConstraintSchema(**request_data)
```

```
        return True, None
    except ValidationError as e:
        return False, str(e)
```

Kiểm Soát Quyền Hạn Thực Thi (Permission Check Guard)

Tại sao việc phân quyền truy cập lại là yếu tố sống còn để đảm bảo an toàn tài chính khi Agent thực hiện thao tác đặt vé?

Nếu Agent có toàn quyền tự động kích hoạt chức năng thanh toán và xuất vé, rủi ro phát sinh giao dịch ngoài ý muốn hoặc bị tấn công câu lệnh (Prompt Injection) là rất lớn. Do đó, lớp bảo vệ quyền hạn (PermissionGuard) được thiết kế nhằm thiết lập cơ chế phân quyền dựa trên ngữ cảnh phiên làm việc (user_session). Hệ thống chia các tác vụ thành hai nhóm rõ rệt: nhóm tác vụ chỉ đọc/giữ chỗ tạm thời và nhóm tác vụ tác động tài chính. Trong khi các hành động tìm kiếm hay đặt giữ chỗ tạm thời được cấp quyền mặc định cho mọi người dùng, tác vụ xác nhận thanh toán (CONFIRM_PAYMENT) bắt buộc phải kiểm tra sự tồn tại của Token xác thực hợp lệ gắn liền với phiên đăng nhập của người dùng.

Bảng ma trận phân quyền tác vụ và điều kiện kích hoạt

Mã tác vụ (Action Type)	Quyền truy cập	Điều kiện kích hoạt (Condition)	Hành vi khi vi phạm
SEARCH	Công khai (Public)	Không yêu cầu xác thực	Cho phép thực thi lập tức.
CHECK_DETAILS	Công khai (Public)	Không yêu cầu xác thực	Cho phép thực thi lập tức.
HOLD_TICKET	Công khai (Public)	Yêu cầu đủ thông tin hành khách	Tạo mã PNR ở trạng thái chờ (HELD).

CONFIRM_PAYMEN T	Hạn chế (Restricted)	Phiên người dùng thuộc danh sách ủy quyền	Từ chối thực thi, yêu cầu cấp quyền hoặc bàn giao.
---------------------	-------------------------	---	---

- Cấu hình trên code như sau:

```
class PermissionGuard:
    """Quản lý và kiểm tra quyền thao tác (đặc biệt đối với tác vụ
    thanh toán/tài chính)."""
    def __init__(self):
        self.authorized_tokens = {"USER_SESSION_8899":
        "AUTH_APPROVED_USER_TOKEN"}

    def is_action_permitted(self, action_type: str, user_session: str)
    -> bool:
        if action_type in ["SEARCH", "CHECK_DETAILS", "HOLD_TICKET"]:
            return True
        if action_type == "CONFIRM_PAYMENT":
            return user_session in self.authorized_tokens
        return False

    def get_auth_token(self, user_session: str) -> Optional[str]:
        return self.authorized_tokens.get(user_session, None)
```

Cơ Chế Bàn Giao Cho Con Người (Handoff Mechanism)

Xử lý thế nào khi Agent rơi vào tình huống bế tắc, không đủ thẩm quyền hoặc gặp sự cố ngoài lệ ngoài tầm xử lý tự động?

Trong quá trình vận hành thực tế, Agent có thể gặp phải các rào cản không thể tự giải quyết như người dùng chưa xác thực thanh toán, hệ thống gặp sự cố không tìm thấy vé, hoặc phát sinh tranh chấp thông tin. Thay vì để Agent lặp vô hạn (Infinite Loop) hoặc trả về phản hồi ảo (Hallucination), lớp quản lý bàn giao (HandoffManager) đóng vai trò là lối thoát an toàn (Fail-safe mechanism). Khi kích hoạt hàm bàn giao, hệ thống sẽ trích xuất toàn bộ ảnh chụp trạng thái hiện tại (state_summary), đóng gói lý do thất bại và

chuyển toàn bộ ngữ cảnh giao dịch sang bộ phận Chăm sóc Khách hàng (Human-in-the-Loop). Điều này đảm bảo tính liên tục của dịch vụ và trải nghiệm người dùng không bị gián đoạn.

- Triển khai trên code như sau:

```
class HandoffManager:
    """Bàn giao quy trình cho nhân viên chăm sóc khách hàng
    (Human-in-the-loop) khi gặp sự cố."""
    @staticmethod
    def trigger_handoff(reason: str, state_summary: Dict[str, Any]) ->
    Dict[str, Any]:
        return {
            "status": "HANDOFF_TO_HUMAN",
            "reason": reason,
            "context": state_summary,
            "action_required": "Yêu cầu chuyên viên CSKH tiếp nhận và
            xử lý thủ công."
        }
```

Tiêu Chí Đánh Giá Hoàn Thành Bằng Code (Code-Based Task Completion Evaluator)

Phương pháp nào giúp đo lường mức độ hoàn thành nhiệm vụ của Agent một cách định lượng, khách quan và hoàn toàn tự động mà không phụ thuộc vào đánh giá cảm quan của LLM?

Để đánh giá chính xác hiệu quả hoạt động của Agent, việc chỉ dựa vào câu trả lời dạng văn bản do LLM sinh ra là không đáng tin cậy. Lớp đánh giá tự động (TaskCompletionEvaluator) áp dụng phương pháp kiểm tra độc lập bằng mã nguồn dựa trên dữ liệu giao dịch thực tế trong cơ sở dữ liệu giả lập (MOCK_BOOKINGS). Lớp này tiếp nhận mã PNR được khởi tạo và đối soát trực tiếp với bộ quy tắc ràng buộc ban đầu của người dùng (BookingConstraintSchema). Đánh giá được tính toán theo thang điểm phạt định lượng từ 0.0 đến 1.0 tùy thuộc vào mức độ vi phạm hoặc thành công của giao dịch.

Bảng tiêu chí và thang điểm đánh giá hoàn thành nhiệm vụ đặt vé

Thang điểm (Score)	Trạng thái kiểm tra	Điều kiện kích hoạt bằng mã nguồn (Code Condition)	Ý nghĩa đánh giá
0.0	Thất bại hoàn toàn	Mã PNR không tồn tại hoặc bị rỗng trong CSDL	Agent không tạo được mã đặt chỗ.
0.3	Chưa hoàn tất	Mã PNR có tồn tại nhưng trạng thái chưa phải CONFIRMED	Giữ chỗ thành công nhưng chưa thanh toán.
0.5	Vi phạm ngân sách	Giá vé thực tế lớn hơn ngân sách tối đa (max_budget)	Đặt vé thành công nhưng vượt quá chi phí cho phép.
0.7	Sai lệch thông tin	Tên hành khách trong vé không khớp với yêu cầu	Thanh toán thành công nhưng sai thông tin người bay.
1.0	Hoàn thiện tuyệt đối	Thỏa mãn tất cả điều kiện: Đã xuất vé, đúng ngân sách, đúng thông tin	Nhiệm vụ hoàn thành hoàn hảo.

- Triển khai trên code như sau:

```
class TaskCompletionEvaluator:
    """Kiểm tra hoàn thành nhiệm vụ tự động bằng mã nguồn (Code-based verification)."""
    @staticmethod
    def evaluate(pnr_code: str, target_constraints:
BookingConstraintSchema) -> Dict[str, Any]:
        if not pnr_code or pnr_code not in MOCK_BOOKINGS:
            return {"success": False, "score": 0.0, "reason": f"Mã PNR '{pnr_code}' không hợp lệ hoặc không tồn tại trong CSDL."}

        booking = MOCK_BOOKINGS[pnr_code]

        # 1. Trạng thái thanh toán
        if booking.get("status") != "CONFIRMED" or not
```

```

booking.get("payment_confirmed"):
    return {"success": False, "score": 0.3, "reason": "Chưa hoàn
tất xác nhận thanh toán."}

# 2. Kiểm tra ràng buộc ngân sách
if booking["price"] > target_constraints.max_budget:
    return {"success": False, "score": 0.5, "reason": f"Giá vé
({booking['price'],} VNĐ) vượt quá ngân sách
({target_constraints.max_budget:,} VNĐ)."}

# 3. Kiểm tra đúng tên người bay
if booking["passenger_name"].upper() !=
target_constraints.passenger_name.upper():
    return {"success": False, "score": 0.7, "reason": "Thông tin
tên hành khách không khớp."}

return {"success": True, "score": 1.0, "reason": "Hoàn thành đặt
vé thành công và đáp ứng đầy đủ tất cả ràng buộc!"}

```

2. Cài đặt Agent với 3 mẫu thiết kế: ReAct, Plan-then-Execute, Lai

Mẫu Thiết Kế 1: ReAct Agent (Reasoning and Acting)

Cơ chế đan xen giữa suy luận (Reasoning) và hành động (Acting) trong mẫu thiết kế ReAct hoạt động như thế nào, và làm sao để kiểm soát rủi ro lặp vô hạn trong đồ thị LangGraph?

Mẫu thiết kế ReAct xây dựng một vòng lặp kín giữa nút suy luận (reason_node) và nút thực thi công cụ (act_node). Tại nút suy luận, Agent phân tích trạng thái tin nhắn hiện tại (AgentState), đánh giá dữ liệu phản hồi từ các bước trước đó để quyết định công cụ cần gọi tiếp theo cùng tham số tương ứng. Khi nhận được câu lệnh gọi công cụ, đồ thị chuyển sang nút thực thi để gọi các hàm nghiệp vụ giả lập và nạp kết quả trả về vào trạng thái chung.

Để ngăn chặn nguy cơ lặp vô hạn do LLM không đưa ra điểm dừng hoặc sự cố hạ tầng, hệ thống tích hợp hai tầng kiểm soát cứng: giới hạn số bước lặp tối đa (MAX_REACT_STEPS = 6) và đối soát trạng thái giao dịch thông qua lớp đánh giá hoàn thành (TaskCompletionEvaluator). Khi mã PNR đã ở trạng thái xác nhận thanh toán (CONFIRMED) hoặc chạm ngưỡng số bước tối đa, nút suy luận tự động ngắt vòng lặp,

tổng hợp toàn bộ lịch sử tương tác và gọi LLM để sinh phản hồi cuối cùng cho hành khách.

Mẫu Thiết Kế 2: Plan-Then-Execute Agent (Lập Kế Hoạch Trước - Thực Thi Sau)

Mẫu thiết kế Plan-then-Execute tái cấu trúc luồng xử lý như thế nào để tối ưu hóa chi phí gọi LLM và tăng tính dự đoán cho quy trình đặt vé?

Khác với cơ chế suy luận từng bước của ReAct, mẫu thiết kế Plan-then-Execute tách biệt hoàn toàn giai đoạn lập kế hoạch và giai đoạn thực thi. Ban đầu, nút lập kế hoạch (`planner_node`) xây dựng một danh sách các bước công việc tĩnh định hình toàn bộ hành trình giao dịch. Nút thực thi (`executor_node`) sau đó đóng vai trò như một bộ điều phối tuyến tính, tự động duyệt qua danh sách kế hoạch dựa trên chỉ số bước hiện tại (`current_step`) để lần lượt kích hoạt các công cụ tương ứng mà không cần triệu gọi LLM ở mỗi vòng lặp trung gian.

Sau khi thực thi hoàn tất tất cả các bước trong kế hoạch, luồng điều khiển chuyển sang nút phản hồi cuối cùng (`final_response_node`). Tại đây, LLM chỉ được kích hoạt một lần duy nhất để tổng hợp dữ liệu giao dịch từ trạng thái hệ thống và sinh câu trả lời hoàn chỉnh. Mẫu thiết kế này giảm thiểu đáng kể số lần gọi API tới mô hình ngôn ngữ, mang lại tốc độ thực thi nhanh và tính ổn định cao, tuy nhiên hạn chế ở khả năng tự điều chỉnh khi một bước trong kế hoạch gặp sự cố ngoài dự kiến.

Mẫu Thiết Kế 3: Hybrid Agent (Mẫu Lai Kết Hợp Dynamic Plan, ReAct và Handoff)

Làm thế nào mẫu kiến trúc Lai (Hybrid) có thể dung hòa giữa tính định hướng của lập kế hoạch, tính linh hoạt của ReAct và yêu cầu an toàn tài chính nhờ cơ chế Bàn giao (Handoff)?

Mẫu kiến trúc Lai được thiết kế nhằm kết hợp ưu điểm của cả hai mô hình trên, đồng thời tích hợp các điểm chặn an toàn (Checkpoints) từ tầng Harness. Nút lập kế hoạch (`planner_node`) định hình một khung kế hoạch chiến lược gồm các giai đoạn nghiệp vụ lớn. Nút thực thi động (`hybrid_react_node`) chịu trách nhiệm xử lý từng giai đoạn, đồng thời trực tiếp kiểm tra quyền hạn thao tác qua lớp kiểm quyền (PermissionGuard) trước khi thực hiện các tác vụ nhạy cảm như thanh toán.

Điểm đổi mới cốt lõi của kiến trúc Lai nằm ở nút điều hướng điều kiện (router). Khi tiến trình xử lý diễn ra bình thường, router duy trì vòng lặp thực thi và chuyển tiếp sang nút hoàn tất (`normal_finish_node`) để LLM gửi lời cảm ơn khách hàng. Tuy nhiên, nếu phát hiện hành vi vi phạm phân quyền (ví dụ: phiên người dùng không có Token thanh toán), router sẽ lập tức rẽ nhánh luồng công việc sang nút bàn giao (`handoff_node`). Tại đây,

Agent trích xuất ngữ cảnh giao dịch hiện tại, thông báo nhẹ nhàng cho khách hàng và chuyển giao hồ sơ xử lý cho nhân viên chăm sóc khách hàng (Human-in-the-Loop).

So Sánh Tổng Quan Sự Khác Biệt Giữa Ba Mẫu Thiết Kế

Những khác biệt cốt lõi về cơ chế điều phối, khả năng thích ứng, chi phí tính toán và mức độ an toàn giữa ba mẫu thiết kế ReAct, Plan-then-Execute và Lai là gì?

Sự khác biệt giữa ba mô hình thể hiện rõ qua cách thức quản lý luồng điều khiển, mức độ can thiệp của LLM trong từng bước thực thi và khả năng phối hợp với các quy tắc bảo vệ của hệ thống.

Bảng so sánh đặc tính kỹ thuật giữa ba mẫu thiết kế Agent

Tiêu chí so sánh	ReAct Agent (ReActFlightAgent)	Plan-then-Execute Agent (PlanThenExecuteFlightAgent)	Hybrid Agent (HybridFlightAgent)
Cơ chế điều phối	Vòng lặp phản hồi đan xen Reason ↔ Act.	Tách biệt: Lập kế hoạch trước, thực thi tuần tự sau.	Khung kế hoạch chiến lược kết hợp thực thi động và phân nhánh điều kiện.
Số lần gọi LLM	Nhiều (gọi LLM ở mỗi vòng lặp suy luận).	Ít (chỉ gọi ở bước đầu lập kế hoạch và bước cuối phản hồi).	Tối ưu (gọi LLM ở bước lập kế hoạch, các bước kết thúc hoặc bàn giao).
Khả năng thích ứng	Rất cao; tự điều chỉnh hành vi dựa trên kết quả trả về của Tool.	Thấp; phụ thuộc vào kế hoạch cố định ban đầu, khó rẽ nhánh.	Cân bằng; tuân thủ quy trình nhưng cho phép rẽ nhánh linh hoạt.
Tích hợp Guardrails / Harness	Kiểm tra điều kiện ngắt ở mỗi vòng lặp reason_node.	Xử lý nối tiếp ở tầng thực thi công cụ.	Trực tiếp tích hợp PermissionGuard và HandoffManager tại router.

Khả năng xử lý sự cố	Dễ rơi vào lặp vô tận nếu không giới hạn bước lặp (MAX_STEPS).	Dễ thất bại toàn chuỗi nếu một bước trung gian phát sinh lỗi.	Tự động chuyển giao cho con người (Handoff) khi gặp lỗi hoặc thiếu quyền.
Trường hợp sử dụng phù hợp	Các bài toán mở, yêu cầu khám phá và tìm kiếm dữ liệu linh hoạt.	Các quy trình cố định, ít biến động, ưu tiên tối ưu chi phí và tốc độ.	Các hệ thống giao dịch tài chính, thương mại điện tử yêu cầu an toàn cao.

So Sánh Các Hàm Nút Thực Thi (Execution Nodes) Trong 3 Mẫu Agent

Các hàm nút chịu trách nhiệm xử lý logic nghiệp vụ và tương tác với các thành phần ngoại vi. Tùy thuộc vào thiết kế kiến trúc, vai trò của từng hàm được phân định cụ thể như sau:

Bảng phân biệt các hàm nút xử lý chính trong 3 kiến trúc Agent

Mẫu thiết kế	Tên hàm (Method)	Nhiệm vụ nghiệp vụ chính	Tần suất gọi LLM	Thay đổi chính trên AgentState
ReAct	reason_node	Phân tích tin nhắn, kiểm tra hoàn thành qua Evaluator, quyết định gọi Tool hoặc kết thúc.	Rất cao (Gọi LLM ở mỗi vòng lặp suy luận).	Cập nhật messages (chứa tool_calls), tăng execution_metrics['steps'].

ReAct	act_node	Trích xuất tên công cụ và tham số từ reason_node để thực thi Tool giả lập.	Không (Chạy code Python thuần).	Cập nhật messages (kết quả Tool), lưu pnr vào State, tăng tool_calls.
Plan-Execute	planner_node	Lập danh sách các bước công việc cố định ban đầu.	Thấp (Khởi tạo kế hoạch tĩnh ban đầu).	Khởi tạo danh sách plan, đặt current_step = 0.
Plan-Execute	executor_node	Duyệt qua chỉ số current_step để thực thi công cụ tương ứng trong kế hoạch.	Không (Gọi trực tiếp hàm Tool theo chỉ số bước).	Tăng current_step, lưu pnr, cập nhật messages và chỉ số execution_metrics.
Plan-Execute	final_response_node	Tổng hợp toàn bộ kết quả thực thi kế hoạch để phản hồi người dùng.	Có (Gọi 1 lần duy nhất khi hoàn thành kế hoạch).	Thêm câu trả lời văn bản hoàn chỉnh của LLM vào messages.
Hybrid	planner_node	Khởi tạo kế hoạch chiến lược dạng danh sách các hành động lớn (SEARCH, HOLD, CONFIRM).	Không (Khai báo khung kế hoạch định sẵn).	Khởi tạo plan, đặt current_step = 0.

Hybrid	hybrid_react_node	Thực thi từng hành động trong kế hoạch, tích hợp kiểm tra quyền qua PermissionGuard.	Không (Gọi công cụ kết hợp kiểm tra điều kiện bảo mật).	Cập nhật current_step, pnr, gán cờ handoff = True nếu thiếu quyền.
Hybrid	handoff_node	Đóng gói ngữ cảnh và yêu cầu LLM soạn thông báo chuyển giao cho nhân viên CSKH.	Có (Gọi 1 lần khi kích hoạt cờ handoff).	Thêm câu thông báo bàn giao sự cố vào messages.
Hybrid	normal_finish_node	Soạn lời cảm ơn và xác nhận giao dịch thành công cho khách hàng.	Có (Gọi 1 lần khi hoàn tất tất cả các bước).	Thêm thông báo hoàn tất giao dịch vào messages.

So Sánh Các Hàm Điều Hướng Luồng (Routing / Conditional Edges)

Các hàm điều hướng đóng vai trò quyết định nút tiếp theo mà đồ thị LangGraph sẽ chuyển tới dựa trên trạng thái thời gian thực của AgentState.

Bảng phân biệt các hàm điều hướng và quy tắc rẽ nhánh trong LangGraph

Mẫu thiết kế	Tên hàm điều hướng	Dữ liệu đầu vào kiểm tra (State Check)	Các nút đích có thể chuyển đến (Target Nodes)
--------------	--------------------	--	---

ReAct	should_continue	Kiểm tra thuộc tính <code>additional_kwargs</code> trong <code>messages[-1]</code> xem có chứa <code>tool_calls</code> hay không.	• "act" (Nếu LLM yêu cầu gọi công cụ). • END (Nếu LLM không gọi thêm công cụ hoặc đạt ngưỡng ngắt).
Plan-Execute	should_continue	So sánh giá trị <code>current_step</code> với tổng độ dài danh sách plan.	• "execute" (Nếu <code>current_step < len(plan)</code>). • "final_response" (Nếu đã duyệt hết kế hoạch).
Hybrid	router	Kiểm tra cờ <code>handoff</code> và chỉ số <code>current_step</code> so với plan.	• "handoff_node" (Nếu <code>handoff == True</code>). • "hybrid_react" (Nếu còn bước trong kế hoạch). • "normal_finish" (Nếu hoàn tất kế hoạch an toàn).

So Sánh Các Hàm Thuộc Lớp Bảo Vệ (Harness) Và Công Cụ (Tools)

Bên cạnh các hàm quản lý nút trong đồ thị Agent, các hàm hỗ trợ thuộc lớp Harness và Tools đóng vai trò thực thi tác vụ chuyên biệt và không tham gia trực tiếp vào luồng điều phối của LangGraph.

Bảng phân biệt các hàm trong lớp Harness và Tool Mockup

Nhóm chức năng	Tên hàm (Function/Method)	Đầu vào (Input Parameters)	Đầu ra (Output Format)	Vai trò chính trong hệ thống
Harness (Bảo vệ)	validate_request	request_data (Dict)	(bool, Optional[str])	Thẩm định tính đúng đắn của dữ liệu đầu vào theo BookingConstraintSchema.
Harness (Bảo vệ)	is_action_permitted	action_type, user_session	bool	Kiểm tra quyền hạn truy cập của người dùng đối với các thao tác nhạy cảm (như thanh toán).
Harness (Bảo vệ)	trigger_handoff	reason, state_summary	Dict[str, Any]	Đóng gói ngữ cảnh giao dịch để chuyển giao hồ sơ cho con người (Human-in-the-Loop).
Harness (Bảo vệ)	evaluate	pnr_code, target_constraints	Dict[str, Any]	Đánh giá định lượng kết quả đặt vé bằng mã nguồn dựa trên CSDL thực tế.
Tool Mock up	search_flights	origin, destination, date	JSON String	Truy vấn danh sách các chuyến bay phù hợp từ CSDL giả lập FLIGHT_DATABASE.
Tool Mock up	hold_ticket	flight_number, passenger_name, passenger_id	JSON String	Giữ chỗ tạm thời, khởi tạo mã PNR và lưu vào MOCK_BOOKINGS.
Tool Mock up	confirm_booking	pnr_code, payment_auth_token	JSON String	Thẩm định Token thanh toán và cập

				nhật trạng thái PNR thành CONFIRMED.
--	--	--	--	---

3.Đánh giá hiệu quả của Agent với 3 mẫu thiết kế khác nhau

Để đánh giá độ hiệu quả của Agent với 3 mẫu thiết kế khác nhau. Ta tiến hành chạy thử và thu thập kết quả như sau.

- Kết quả REACT AGENT.

```
Admin@Admin MINGW64 /f/class/HI_Tích hợp AI/23520195_DuongChiCuong_SE373.R11_TH (main)
$ python -m lab03.main
=====
🚀 BẮT ĐẦU CHẠY THỬ NGHIỆM 3 MẪU THIẾT KẾ AGENT
=====
[1] TEST REACT AGENT...

[VÒNG LẶP / BƯỚC: ReAct Step 1]
🤖 Model đang sử dụng : gemini-3.5-flash-lite
🔧 Hàm được gọi (Tool) : search_flights
📄 Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: ReAct Step 2]
🤖 Model đang sử dụng : gemini-3.5-flash-lite
🔧 Hàm được gọi (Tool) : hold_ticket
📄 Tham số (Parameters): {"flight_number": "VN123", "passenger_name": "Nguyen Van A", "passenger_id": "0123456789"}

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

=====
📄 KẾT QUẢ HỘI THOẠI & THỰC THI: REACT AGENT
=====
👤 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

📊 Thông kê thực thi:
• Số bước suy luận / thực thi : 3
• Tổng số lần gọi Tool : 2
• Mã đặt chỗ (PNR) : PNR-159713
• Kích hoạt bàn giao CSKH : False

🤖 Agent:
Calling confirm_booking
=====
```

=====

=====

 **BẮT ĐẦU CHẠY THỬ NGHIỆM 3 MẪU THIẾT KẾ AGENT**

=====

=====

[1] TEST REACT AGENT...

— [VÒNG LẶP / BƯỚC: ReAct Step 1]

 Model đang sử dụng : gemini-3.5-flash-lite

 Hàm được gọi (Tool) : search_flights

 Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: ReAct Step 2]



Model đang sử dụng : gemini-3.5-flash-lite

Hàm được gọi (Tool) : hold_ticket

Tham số (Parameters): {"flight_number": "VN123", "passenger_name": "Nguyen Van A", "passenger_id": "0123456789"}

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

🗨️ KẾT QUẢ HỘI THOẠI & THỰC THI: REACT AGENT

 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

 Thống kê thực thi:

- Số bước suy luận / thực thi : 3
- Tổng số lần gọi Tool : 2
- Mã đặt chỗ (PNR) : PNR-159713
- Kích hoạt bàn giao CSKH : False



Agent:

Calling confirm_booking

- Kết quả PLAN-THEN-EXECUTE AGENT.

```
-----
[2] TEST PLAN-THEN-EXECUTE AGENT...

[VÒNG LẶP / BƯỚC: Plan Step 1] -----
🤖 Model đang sử dụng : gemini-3.5-flash-lite
🔧 Hàm được gọi (Tool) : search_flights
📄 Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: Plan Step 2] -----
🤖 Model đang sử dụng : gemini-3.5-flash-lite
🔧 Hàm được gọi (Tool) : hold_ticket
📄 Tham số (Parameters): {"flight_number": "VN123", "passenger_name": "Nguyen Van A", "passenger_id": "0123456789"}

[VÒNG LẶP / BƯỚC: Plan Step 3] -----
🤖 Model đang sử dụng : gemini-3.5-flash-lite
🔧 Hàm được gọi (Tool) : confirm_booking
📄 Tham số (Parameters): {"pnr_code": "PNR-164281", "payment_auth_token": "AUTH_APPROVED_USER_TOKEN"}

=====
📄 KẾT QUẢ HỘI THOẠI & THỰC THI: PLAN-THEN-EXECUTE AGENT
=====
👤 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

📊 Thống kê thực thi:
• Số bước suy luận / thực thi : 3
• Tổng số lần gọi Tool      : 3
• Mã đặt chỗ (PNR)         : PNR-164281
• Kích hoạt bàn giao CSKH   : False

👤 Agent:
Kính chào quý khách **Nguyen Van A**,

Tôi xin trân trọng thông báo quý khách đã đặt vé máy bay thành công. Dưới đây là thông tin chi tiết về hành trình của quý khách:

* **Mã đặt chỗ (PNR):** PNR-164281
* **Hành trình:** TP. Hồ Chí Minh (SGN) đi Hà Nội (HAN)
* **Ngày khởi hành:** 10/10/2026
* **Chuyến bay:** VN123 (Vietnam Airlines)
* **Giờ bay:** 08:00 - 10:10
* **Hạng vé:** Phổ thông (Economy)
* **Trạng thái:** Đã thanh toán và xác nhận xuất vé thành công

Cảm ơn quý khách đã tin tưởng và sử dụng dịch vụ đặt vé của chúng tôi. Chúc quý khách có một chuyến đi an toàn và tốt đẹp!

Nếu quý khách cần hỗ trợ thêm thông tin gì, xin vui lòng cho tôi biết nhé.
=====
-----
```

[2] TEST PLAN-THEN-EXECUTE AGENT...

[VÒNG LẶP / BƯỚC: Plan Step 1]

 Model đang sử dụng : gemini-3.5-flash-lite

 Hàm được gọi (Tool) : search_flights

 Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: Plan Step 2]

 Model đang sử dụng : gemini-3.5-flash-lite

 Hàm được gọi (Tool) : hold_ticket

 Tham số (Parameters): {"flight_number": "VN123", "passenger_name":

"Nguyen Van A", "passenger_id": "0123456789"}

— [VÒNG LẶP / BƯỚC: Plan Step 3]

 Model đang sử dụng : gemini-3.5-flash-lite
 Hàm được gọi (Tool) : confirm_booking
 Tham số (Parameters): {"pnr_code": "PNR-164281", "payment_auth_token": "AUTH_APPROVED_USER_TOKEN"}

=====
==
💬 KẾT QUẢ HỘI THOẠI & THỰC THI: PLAN-THEN-EXECUTE AGENT
=====

=====
 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

 Thống kê thực thi:
• Số bước suy luận / thực thi : 3
• Tổng số lần gọi Tool : 3
• Mã đặt chỗ (PNR) : PNR-164281
• Kích hoạt bàn giao CSKH : False

 Agent:
Kính chào quý khách **Nguyen Van A**,

Tôi xin trân trọng thông báo quý khách đã đặt vé máy bay thành công. Dưới đây là thông tin chi tiết về hành trình của quý khách:

* **Mã đặt chỗ (PNR):** `PNR-164281`
* **Hành trình:** TP. Hồ Chí Minh (SGN) đi Hà Nội (HAN)
* **Ngày khởi hành:** 10/10/2026
* **Chuyến bay:** VN123 (Vietnam Airlines)
* **Giờ bay:** 08:00 - 10:10
* **Hạng vé:** Phổ thông (Economy)
* **Trạng thái:** **Đã thanh toán và xác nhận xuất vé thành công**

Cảm ơn quý khách đã tin tưởng và sử dụng dịch vụ đặt vé của chúng tôi. Chúc quý khách có một chuyến đi an toàn và tốt đẹp!

Nếu quý khách cần hỗ trợ thêm thông tin gì, xin vui lòng cho tôi biết nhé.

=====

=====

=====

● Kết quả HYBRID AGENT (Có Handoff).

```
[3] TEST HYBRID AGENT (Có Handoff)...

[VÒNG LẶP / BƯỚC: Hybrid Step 1]
Model đang sử dụng : gemini-3.5-Flash-lite
Hàm được gọi (Tool) : search_Flights
Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: Hybrid Step 2]
Model đang sử dụng : gemini-3.5-Flash-lite
Hàm được gọi (Tool) : hold_ticket
Tham số (Parameters): {"flight_number": "VN123", "passenger_name": "Nguyen Van A", "passenger_id": "0123456789"}

[VÒNG LẶP / BƯỚC: Hybrid Step 3]
Model đang sử dụng : gemini-3.5-Flash-lite
Hàm được gọi (Tool) : TRIGGER_HANDOFF
Tham số (Parameters): {"reason": "Permission Denied"}

=====
🗨️ KẾT QUẢ HỎI THOẠI & THỰC THI: HYBRID AGENT
=====
👤 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

📊 Thông kê thực thi:
• Số bước suy luận / thực thi : 3
• Tổng số lần gọi Tool : 2
• Mã đặt chỗ (PNR) : PNR-169241
• Kích hoạt bàn giao CSKH : True

🤖 Agent:
Chào anh/chị,

Em đã hỗ trợ tìm kiếm và giữ chỗ thành công chuyến bay từ **SGN đi HAN** ngày **2026-10-10** cho hành khách **Nguyen Van A** với thông tin như sau:

* **Mã giữ chỗ (PNR):** "PNR-169241"
* **Chuyến bay:** VN123 (Vietnam Airlines)
* **Giờ khởi hành:** 08:00 - 10:10
* **Giá vé:** 2.100.000 VND

Tuy nhiên, hệ thống thanh toán tự động hiện tại đang cần phân quyền thêm cho tài khoản. Do đó, em đã chuyển tiếp yêu cầu giữ chỗ này trực tiếp đến bộ phận Chăm sóc Khách hàng (CSKH).

Nhân viên CSKH sẽ liên hệ lại với anh/chị trong thời gian sớm nhất để hỗ trợ hoàn tất thủ tục xuất vé. Anh/chị vui lòng để ý điện thoại giúp em nhé ạ!

Em cảm ơn anh/chị đã sử dụng dịch vụ!

=====
(venv)
Admin@Admin MINGW64 /F/class/HT_Tích hxp AI/23520195_DuongChiCuong_SE373.R11_TH (main)
C:\>git add
```

[3] TEST HYBRID AGENT (Có Handoff)...

└── [VÒNG LẶP / BƯỚC: Hybrid Step 1]



Model đang sử dụng : gemini-3.5-flash-lite



Hàm được gọi (Tool) : search_flights



Tham số (Parameters): {"origin": "SGN", "destination": "HAN", "date": "2026-10-10"}

[VÒNG LẶP / BƯỚC: Hybrid Step 2]

 Model đang sử dụng : gemini-3.5-flash-lite
 Hàm được gọi (Tool) : hold_ticket
 Tham số (Parameters): {"flight_number": "VN123", "passenger_name": "Nguyen Van A", "passenger_id": "0123456789"}

[VÒNG LẶP / BƯỚC: Hybrid Step 3]

 Model đang sử dụng : gemini-3.5-flash-lite
 Hàm được gọi (Tool) : TRIGGER_HANDOFF
 Tham số (Parameters): {"reason": "Permission Denied"}

====
💬 KẾT QUẢ HỘI THOẠI & THỰC THI: HYBRID AGENT
=====

====
 User: "Tôi muốn tìm vé và đặt vé từ SGN đi HAN vào ngày 2026-10-10 cho hành khách Nguyen Van A."

 Thống kê thực thi:

- Số bước suy luận / thực thi : 3
- Tổng số lần gọi Tool : 2
- Mã đặt chỗ (PNR) : PNR-169241
- Kích hoạt bàn giao CSKH : True

 Agent:
Chào anh/chị,

Em đã hỗ trợ tìm kiếm và giữ chỗ thành công chuyến bay từ **SGN đi HAN** ngày **2026-10-10** cho hành khách **Nguyen Van A** với thông tin như sau:

* **Mã giữ chỗ (PNR):** `PNR-169241`
* **Chuyến bay:** VN123 (Vietnam Airlines)
* **Giờ khởi hành:** 08:00 - 10:10
* **Giá vé:** 2.100.000 VNĐ

Tuy nhiên, hệ thống thanh toán tự động hiện tại đang cần phân quyền thêm cho tài khoản. Do đó, em đã chuyển tiếp yêu cầu giữ chỗ này trực tiếp đến bộ phận Chăm sóc khách hàng (CSKH).

Nhân viên CSKH sẽ liên hệ lại với anh/chị trong thời gian sớm nhất để hỗ trợ hoàn tất thủ tục xuất vé. Anh/chị vui lòng để ý điện thoại giúp em nhé ạ!

Em cảm ơn anh/chị đã sử dụng dịch vụ!

=====

(venv)
Admin@Admin MINGW64 /f/class/HT_Tích hợp
AI/23520195_DuongChiCuong_SE373.R11_TH (main)

Đánh Giá Chi Tiết Theo Từng Mẫu Thiết Kế

ReAct Agent (ReActFlightAgent) — Thất bại ở bước cuối / Dừng dở dang

- Thông số thực thi:
 - Số bước: 3 bước | Số lần gọi Tool: 2 lần (search_flights, hold_ticket).
 - Mã PNR: PNR-159713 (Mới chỉ giữ chỗ thành công).
 - Cờ Handoff: False.
- Phân tích kết quả:
 - Ưu điểm: Tiến hành tìm kiếm và tạo mã giữ chỗ nhanh chóng ở 2 bước đầu.
 - Hạn chế nghiêm trọng: Agent bị "kẹt" ở bước 3. Khi đưa ra quyết định gọi confirm_booking, hệ thống ngắt vòng lặp hoặc trả về chuỗi văn bản thô: "Calling confirm_booking" cho người dùng thay vì thực thi Tool và tổng hợp câu trả lời hoàn chỉnh.
 - Nguyên nhân: Do đặc thù vòng lặp ReAct phụ thuộc hoàn toàn vào phản hồi tin nhắn của LLM. Khi dữ liệu tin nhắn trả về chứa tool_calls, luồng xử lý bị cắt ngang do điều kiện dừng hoặc lỗi AFC (Automatic Function Calling) từ Gemini API, dẫn đến trải nghiệm người dùng (UX) bị lỗi.

Plan-Then-Execute Agent (PlanThenExecuteFlightAgent) — Hoàn hảo / Tối ưu cho quy trình cố định

- Thông số thực thi:

- Số bước: 3 bước | Số lần gọi Tool: 3 lần (search_flights, hold_ticket, confirm_booking).
- Mã PNR: PNR-164281 (Đã xác nhận & thanh toán thành công).
- Cờ Handoff: False.
- Phân tích kết quả:
 - Ưu điểm: Thực thi tuân thủ 100% kế hoạch ban đầu mà không bị chệch hướng. Phản hồi cuối cùng qua nút final_response_node được trình bày định dạng Markdown xuất sắc, đầy đủ thông tin (PNR, hành trình, giờ bay, hạng vé, trạng thái thanh toán).
 - Hạn chế: Chỉ hoạt động tốt trong kịch bản lý tưởng (đã được cấp Token thanh toán hợp lệ AUTH_APPROVED_USER_TOKEN). Nếu phát sinh lỗi ở bước trung gian, mẫu này khó rẽ nhánh để xử lý linh hoạt.

Hybrid Agent (HybridFlightAgent) — An toàn tuyệt đối / Xử lý ngoại lệ xuất sắc

- Thông số thực thi:
 - Số bước: 3 bước | Số lần gọi Tool: 2 lần (search_flights, hold_ticket) + 1 lần kích hoạt TRIGGER_HANDOFF.
 - Mã PNR: PNR-169241 (Đã giữ chỗ, chưa thanh toán).
 - Cờ Handoff: True (Lý do: "Permission Denied").
- Phân tích kết quả:
 - Ưu điểm: Thể hiện rõ sức mạnh của tầng an toàn (Harness Layer). Khi tiến tới bước thanh toán, Agent kiểm tra quyền qua PermissionGuard và phát hiện tài khoản chưa được phân quyền thanh toán tự động. Lập tức, Router kích hoạt handoff_node để rẽ nhánh sang quy trình xử lý sự cố.
 - Trải nghiệm người dùng: Phản hồi lịch sự, giải thích rõ ràng rằng đã giữ chỗ thành công mã PNR-169241 với giá 2.100.000 VNĐ, đồng thời thông báo chuyển giao cho nhân viên CSKH hỗ trợ tiếp. Đây là cơ chế Fail-safe (sự cố an toàn) tiêu chuẩn cho ứng dụng thực tế.

Bảng So Sánh Hiệu Quả Thực Tế Qua Log Kiểm Thử

Tiêu chí đánh giá	ReAct Agent	Plan-Then-Execute Agent	Hybrid Agent
-------------------	-------------	-------------------------	--------------

Mức độ hoàn thành quy trình	33% (Chỉ giữ chỗ, lỗi xuất vé)	100% (Xuất vé thành công)	100% (Xử lý an toàn theo luồng Handoff)
Số công cụ thực thi thành công	2 / 3 tools	3 / 3 tools	2 tools + 1 Handoff Trigger
Tính an toàn & Bảo mật	Thấp (Dễ ngắt luồng không kiểm soát)	Trung bình (Đòi hỏi dữ liệu đầu vào chuẩn)	Rất cao (Ngăn chặn thao tác trái phép)
Chất lượng phản hồi người dùng	Kém ("Calling confirm_booking")	Rất tốt (Đầy đủ, đẹp mắt, chính xác)	Xuất sắc (Khéo léo, minh bạch, đúng ngữ cảnh)
Khả năng ứng dụng sản phẩm	Cần cải thiện xử lý Exception	Tốt cho các API tĩnh	Tối ưu nhất cho Production

Đánh Giá Tổng Kết

- Plan-Then-Execute Agent đạt hiệu năng cao nhất cho các tác vụ chuẩn hóa, giúp tiết kiệm chi phí gọi LLM và sinh ra câu trả lời cho người dùng hoàn hảo nhất khi mọi điều kiện đều thuận lợi.
- Hybrid Agent là kiến trúc mạnh mẽ và đáng tin cậy nhất. Việc kết hợp Lập kế hoạch + Kiểm soát quyền hạn (PermissionGuard) + Bàn giao con người (Handoff) giúp hệ thống vừa tự động hóa được các bước an toàn (tìm kiếm, giữ chỗ), vừa ngăn chặn được rủi ro tài chính khi gặp thao tác nhạy cảm.
- ReAct Agent trong bài thử nghiệm này bộc lộ nhược điểm về độ ổn định khi tích hợp với thư viện API mới, dễ gặp lỗi ngắt vòng lặp giữa chừng nếu không có bộ điều phối State chặt chẽ.

TÀI LIỆU THAM KHẢO

- [1]. [1] Harrison Chase, "LangChain: Building applications with LLMs through composability," LangChain Documentation, 2023. [Online]. Available: [\[https://python.langchain.com/\]](https://python.langchain.com/)(<https://python.langchain.com/>)
- [2]. [2] LangChain Team, "LangGraph: Building Stateful, Multi-Actor Applications with LLMs," LangChain Blog, 2024. [Online]. Available: [\[https://blog.langchain.dev/langgraph/\]](https://blog.langchain.dev/langgraph/)(<https://blog.langchain.dev/langgraph/>)
- [3]. [3] L. Weng, "LLM Powered Autonomous Agents," lilianweng.github.io, Jun. 2023. [Online]. Available: [\[https://lilianweng.github.io/posts/2023-06-23-agent/\]](https://lilianweng.github.io/posts/2023-06-23-agent/)(<https://lilianweng.github.io/posts/2023-06-23-agent/>)
- [4].